Project 2

Simon Christensen

Section 1: Data Description

```
library(tidymodels)
library(tidyverse)
library(broom)
library(cluster)
library(ISLR2)
library(rsample)
library(LearnPCA)
library(Lahman)
library(mvtnorm)
library(naniar)
library(softImpute)
library(mice)
library(GGally)
library(tidyclust)
library(dendextend)
library(mclust)
library(factoextra)
library(kknn)
library(forcats)
library(MASS)
library(e1071)
library(klaR)
library(discrim)
library(baseballr)
library(dplyr)
```

```
batstats <- readr::read_csv(here::here("batstats.csv"))
```

```
Rows: 448 Columns: 22
-- Column specification --------------------------------------------------------
Delimiter: ","
chr  (1): last_name, first_name
dbl (21): player_id, year, player_age, ab, pa, hit, single, double, triple, ...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

I acquired my data off of Baseball Savant, a site dedicated to tracking every single baseball metric there is. For this project, I set up a data frame containing the basic stats of every hitter in 2018 with over 100 plate appearances in order for each player to have a big enough contribution to the data. Every observation in this data set represents a player. The stats, or variables, that are in the data set are at bats, plate appearances, hits, singles, doubles, triples, home runs, strikeouts, walks, batting average, slugging percentage, on-base percentage, on-base-plus-slugging percentage, runs batted in, total bases, runs, and games played. The data is collected by Baseball Savant from all games where the players participated in order to see how the players' abilities are improving or declining.

The below chunk involves renaming the variables of my batting_2018 data set to my liking as well adding a new variable called AllStar. This data was taken from the AllStarFull data set under the Lahman package. I cleaned the data so that the names of my data set and the names of the AllStarFull data set would match. If the names present in my data set are present in the cleaned AllStarFull data set, a yes will appear in the new AllStar column in the batting_2018 data set, otherwise a no will appear. No data, whether it be the data I got from Baseball Savant or the data already in the AllStarFull set, was altered, only cleaned in order to combine them proper

```
batting_2018 <- batstats |>
  rename(
  Name="last_name, first_name",
  age=player_age,
  hr=home_run,
  k=strikeout,
  bb=walk,
  avg=batting_avg,
  slg=slg_percent,
  obp=on_base_percent,
  ops=on_base_plus_slg,
  rbi=b_rbi,
  tb=b_total_bases,
  g=b_game,
  r=r_run,
```

```r
  sb=r_total_stolen_base
) |>
  mutate(
    Name = str_replace_all(Name, "\\*", ""),
    Name = str_replace_all(Name, ",", ""),
    Name = str_squish(Name),
    Name = sapply(strsplit(Name, " "), function(x) {
      paste(rev(x), collapse = " ")
    }),
    Name = str_to_lower(Name),
    Name = str_replace_all(Name, "[[:punct:]]", ""),
    Name = str_replace_all(Name, "\\s+", ""))
allstars <- AllstarFull |>
  dplyr::select(playerID, yearID)
people <- People |>
  dplyr::select(playerID, nameFirst, nameLast) |>
  dplyr::mutate(Name = paste(nameFirst, nameLast))
allstars_named <- allstars |>
  filter(yearID == 2018) |>
  left_join(people, by = "playerID") |>
  dplyr::select(Name, yearID) |>
  distinct()
clean_name <- function(x) {
  x |>
    str_to_lower() |>
    str_replace_all("[[:punct:]]", "") |>
    str_replace_all("\\s+", "")
}
allstars_named <- allstars_named |>
  mutate(Name = clean_name(Name))
allstar_names <- unique(allstars_named$Name)
batting_2018 <- batting_2018 |>
  mutate(AllStar = ifelse(Name %in% allstar_names, "Yes", "No"))
```

```r
batting_2018$AllStar <- as.factor(batting_2018$AllStar)
set.seed(1)
batting_split <- initial_split(batting_2018,prop=0.8)
bat_train <- training(batting_split)
bat_test <- testing(batting_split)
```

With 448 observations in my data set, I split my data 70%/30% between my training set and my test set respectively in order for my training data to properly predict value on my test set.

I also wanted to ensure that my test set had enough observations in the 'AllStar' category in order to having proper and interesting predictions.

Section 2: Data Wrangling and Exploratory Data Analysis

The below chunk will summarize two variables, home runs and batting average, separately and produce graphs that will show the distribution of each variable.

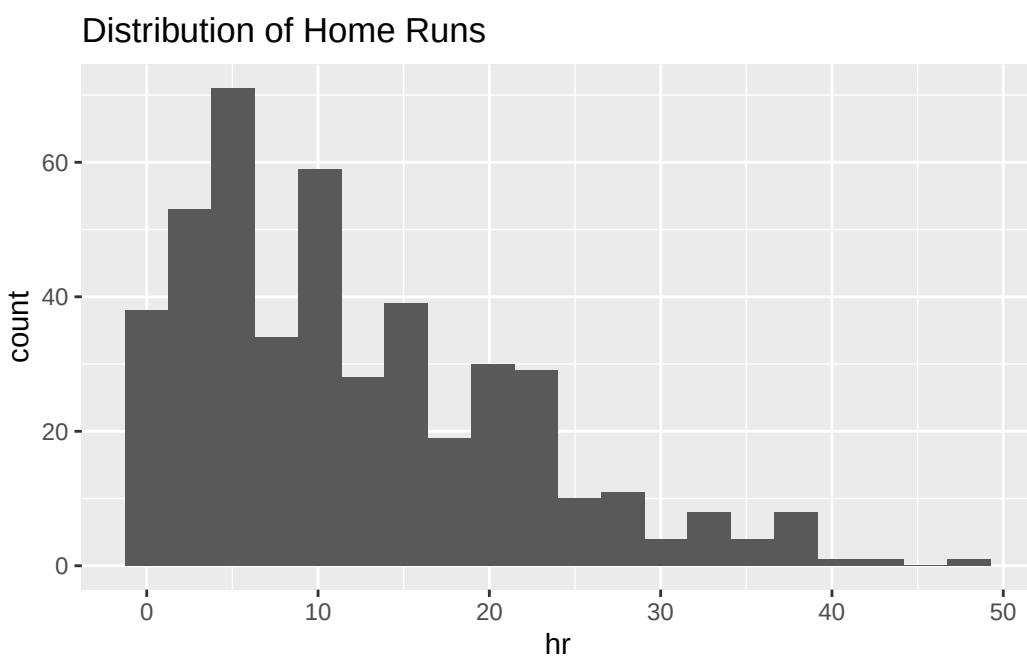
```
summary(batting_2018$hr)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
    0.0     4.0    10.0    12.1    18.0    48.0 
```

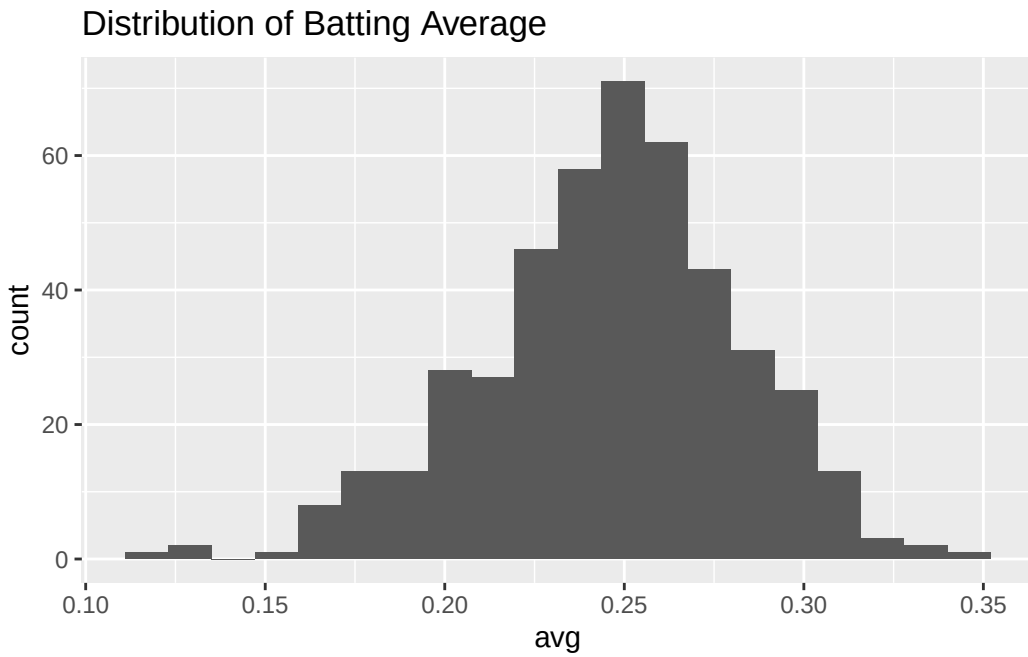
```
summary(batting_2018$avg)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max. 
 0.1170  0.2248  0.2480  0.2461  0.2692  0.3460 
```

```
ggplot(batting_2018, aes(x = hr)) +
  geom_histogram(bins = 20) +
  labs(title = "Distribution of Home Runs")
```



```r
ggplot(batting_2018, aes(x = avg)) +
  geom_histogram(bins = 20) +
  labs(title = "Distribution of Batting Average")
```

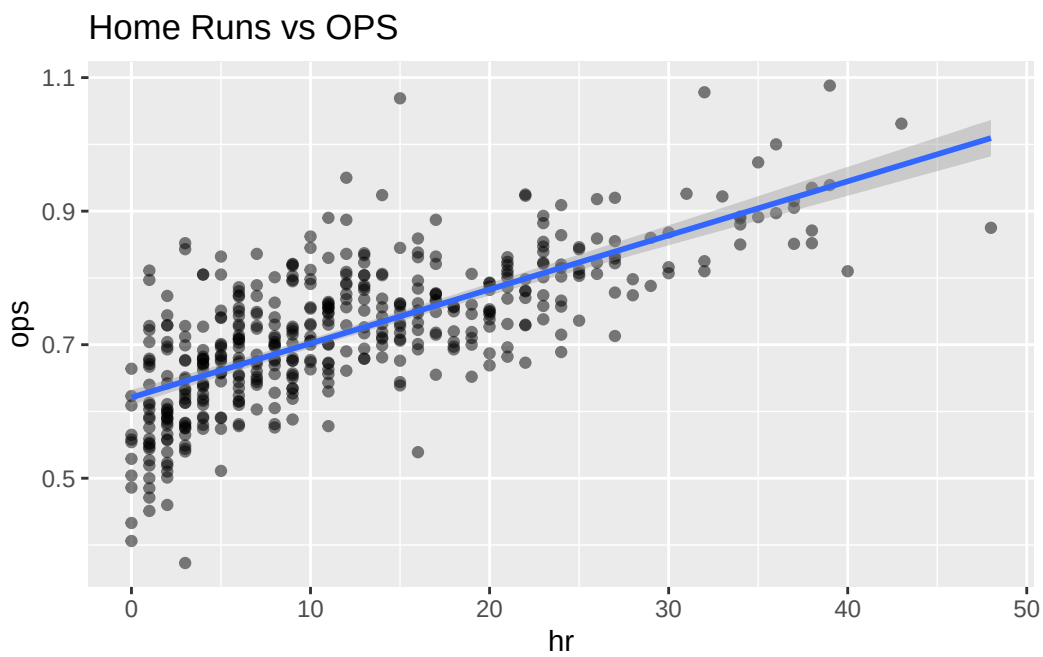


As you can see, the distribution of home runs is skewed right as only a few star players are able to hit above 30 home runs. However, the batting average variable has an almost-perfectly normal distribution which is expected.

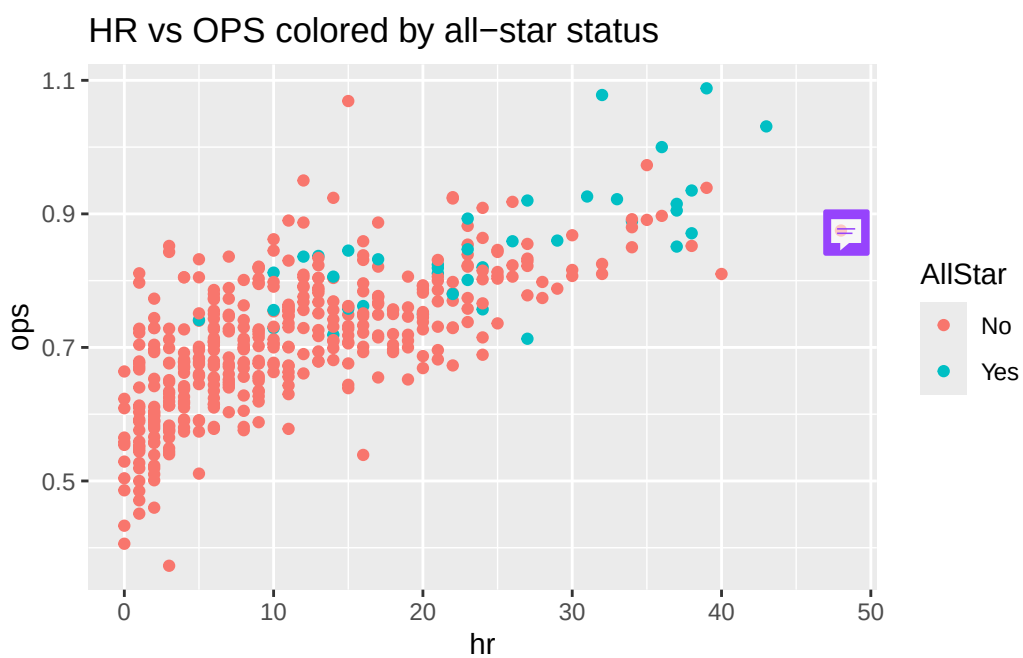
The below chunk will produce a bivariate graph showing the relationship between home runs and OPS as well as a multivariate graph showing the relationship between the two variables while highlighting each players' all star status.

```r
ggplot(batting_2018, aes(x = hr, y = ops)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm") +
  labs(title = "Home Runs vs OPS")
```

```
`geom_smooth()` using formula = 'y ~ x'
```




```
ggplot(batting_2018, aes(x = hr, y = ops, color = AllStar)) +
  geom_point() +
  labs(title = "HR vs OPS colored by all-star status")
```



The variables home runs and OPS are heavily correlated as OPS is calculated using total bases, which home runs has a big contribution towards. So a player who hits a lot of home runs would likely have a higher OPS than a contact hitter with very little power. This results in home runs and OPS having a clear, positive correlation. When considering the multivariate graph, it is interesting that no player who was an all star has an OPS below 0.700. This result will become more apparent and important later.

Some questions I have that pertain to my data are as follows:

1. What variables are more important when it comes to predicting a player's all-star status?
2. Which variables are heavily correlated with each other? Which ones are not?
3. Are the variables that I selected for hitters reliable in predicting all-star status?
4. What variables are most telling of how a player performs at the plate in general?

Section 3: Unsupervised Learning

```
batting_stats <- bat_train |>
  transmute(
    Name=Name,
    avg=avg,
    slg=slg,
    obp=obp,
    ops=ops,
    bbrate = bb/pa,
    krate = k/pa,
    singlerate=single/pa,
    doublerate = double/pa,
    triplerate = triple/pa,
    hrrate = hr/pa,
    rrate = r/pa,
    rbirate = rbi/pa,
    tbrate=tb/pa,
    grate=g/pa,
    sbrate=sb/pa
  )
```

```
batting_recipe <- recipe(
  ~.,data=batting_stats
) |>
  update_role(Name,new_role="id") |>
  step_normalize(all_numeric_predictors()) |>
```

```r
  step_pca(all_predictors(),num_comp = 5)
batting_prep <- batting_recipe |> prep()
batting_bake <- batting_prep |> bake(new_data=NULL)
batting_bake
```

```
# A tibble: 358 x 6
   Name             PC1       PC2     PC3     PC4    PC5
   <chr>          <dbl>     <dbl>   <dbl>   <dbl>  <dbl>
 1 nelsoncruz     -2.88  1.83      0.730   0.0813 -1.35
 2 tylerwhite     -3.67  1.03     -0.0456  0.501   0.499
 3 ronnyrodríguez  2.60  0.000414  1.32    0.759  -0.423
 4 aaronjudge     -4.10  1.79     -0.843  -1.75   -1.03
 5 jeansegura     -1.55 -3.50      0.0210  0.629  -0.763
 6 joséramírez    -4.93  0.533    -2.24   -1.02    0.317
 7 adameaton      -1.82 -2.61     -0.481  -1.46   -0.538
 8 ajosémartínez  -2.46 -1.36      1.87   -0.458  -0.713
 9 ryanohearn     -5.02  3.05     -0.714   0.669   0.948
10 sandyleón       4.18  1.28      0.798   0.537   0.765
# i 348 more rows
```

After looking at a scree plot, I chose to compute 5 principal components for each player as these five components alone explain about 85% of the variance in the data.

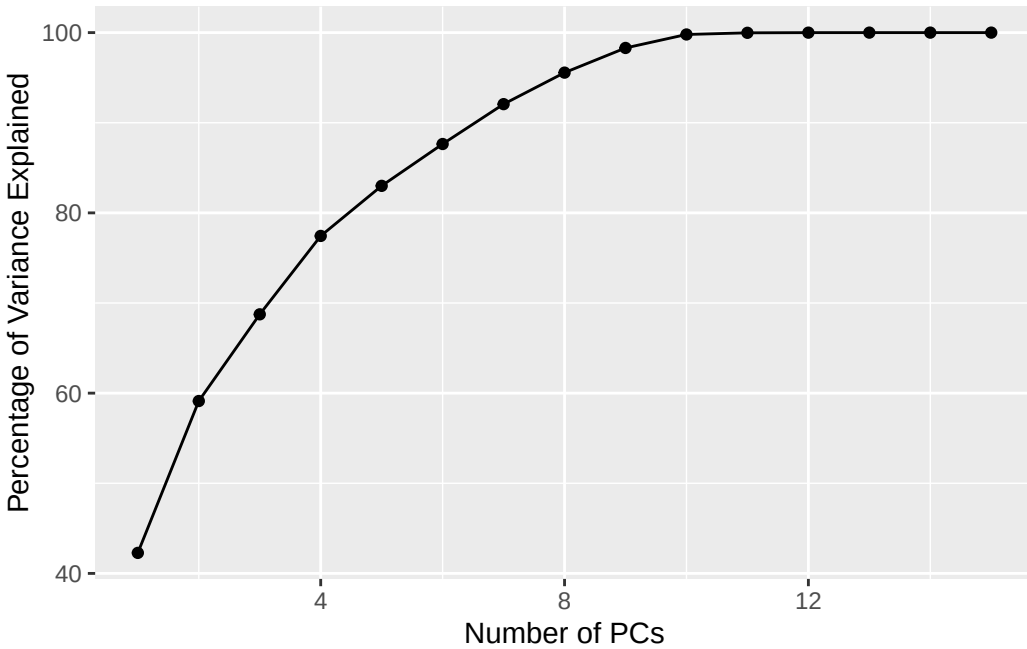
```r
loadings <- tidy(batting_prep,2,type="coef")
pve <- tidy(batting_prep,2,type="variance")
loadings
```

```
# A tibble: 225 x 4
   terms        value component id
   <chr>        <dbl> <chr>     <chr>
 1 avg        -0.314  PC1       pca_0SzNb
 2 slg        -0.385  PC1       pca_0SzNb
 3 obp        -0.317  PC1       pca_0SzNb
 4 ops        -0.391  PC1       pca_0SzNb
 5 bbrate     -0.112  PC1       pca_0SzNb
 6 krate       0.104  PC1       pca_0SzNb
 7 singlerate -0.0656 PC1       pca_0SzNb
 8 doublerate -0.221  PC1       pca_0SzNb
 9 triplerate -0.0165 PC1       pca_0SzNb
10 hrrate     -0.262  PC1       pca_0SzNb
# i 215 more rows
```

```r
pve
```

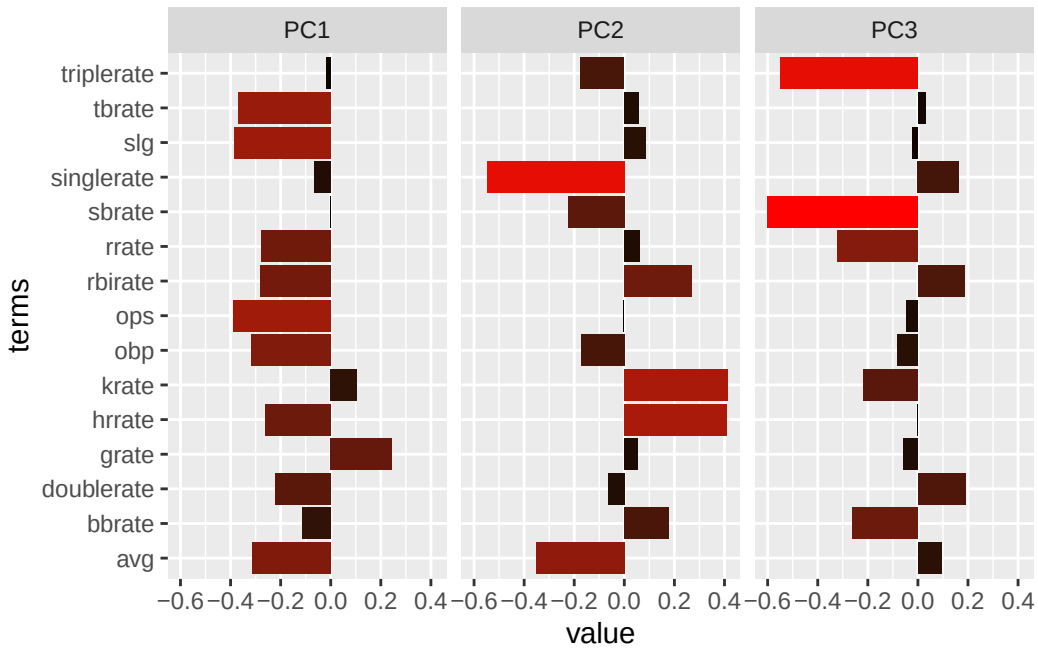
```
# A tibble: 60 x 4
   terms    value component id
   <chr>    <dbl>     <int> <chr>
 1 variance 6.34          1 pca_0SzNb
 2 variance 2.53          2 pca_0SzNb
 3 variance 1.44          3 pca_0SzNb
 4 variance 1.30          4 pca_0SzNb
 5 variance 0.834         5 pca_0SzNb
 6 variance 0.696         6 pca_0SzNb
 7 variance 0.664         7 pca_0SzNb
 8 variance 0.524         8 pca_0SzNb
 9 variance 0.411         9 pca_0SzNb
10 variance 0.224        10 pca_0SzNb
# i 50 more rows
```

```r
ggplot(
  data=pve |> filter(terms=="cumulative percent variance"),
  mapping=aes(x=component,y=value)
) +
  geom_point() + geom_line() +
  labs(
    x="Number of PCs",
    y="Percentage of Variance Explained"
  )
```




```
#|label: plot the loadings to see what each PC is doing

loadings |> filter(component %in% c("PC1","PC2","PC3")) |>
  ggplot(
    mapping=aes(x=value,y=terms,fill=abs(value))
  ) +
  geom_col() + #horizontal bar chart
  theme(legend.position="none") +
  scale_fill_gradient(low="black",high="red") +
  facet_wrap(vars(component)) # PC1 and PC2 on separate graphs
```



PC 1: Represents all-around offensive ability, showcasing a low strikeout rate combined with high offensive stats such as total base rate, slugging percentage, OPS, and OBP just to name a few.

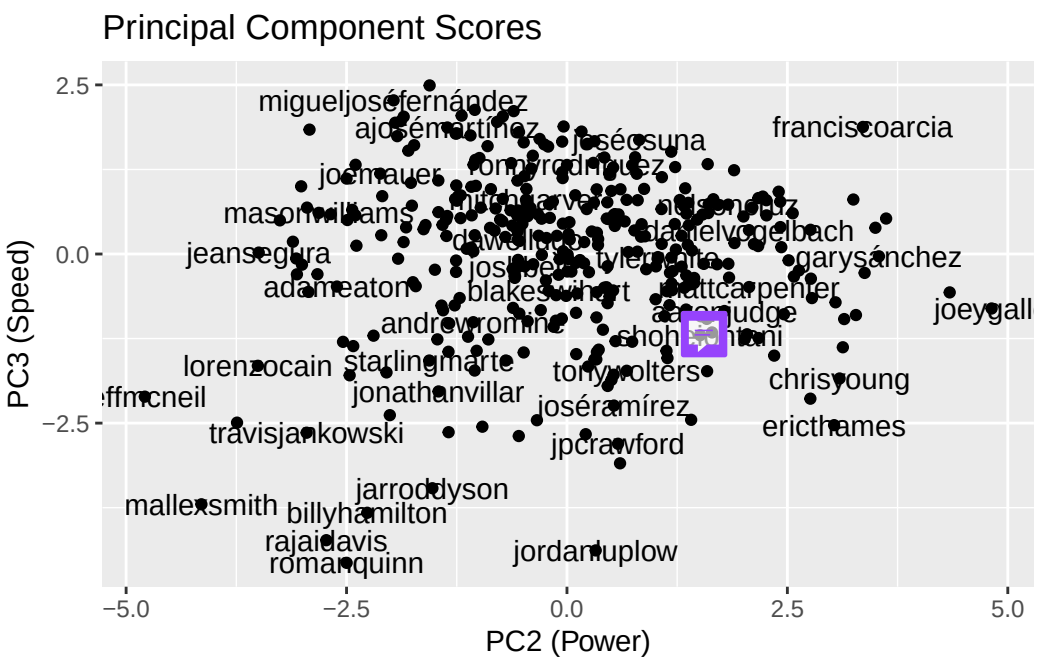
PC 2: Showcases a high home run, single, and strikeout rate, making this principal component a representative of power hitters.

PC3: Represents speed and scoring ability since this principal component equates to a higher triple, stolen base, and run rate.

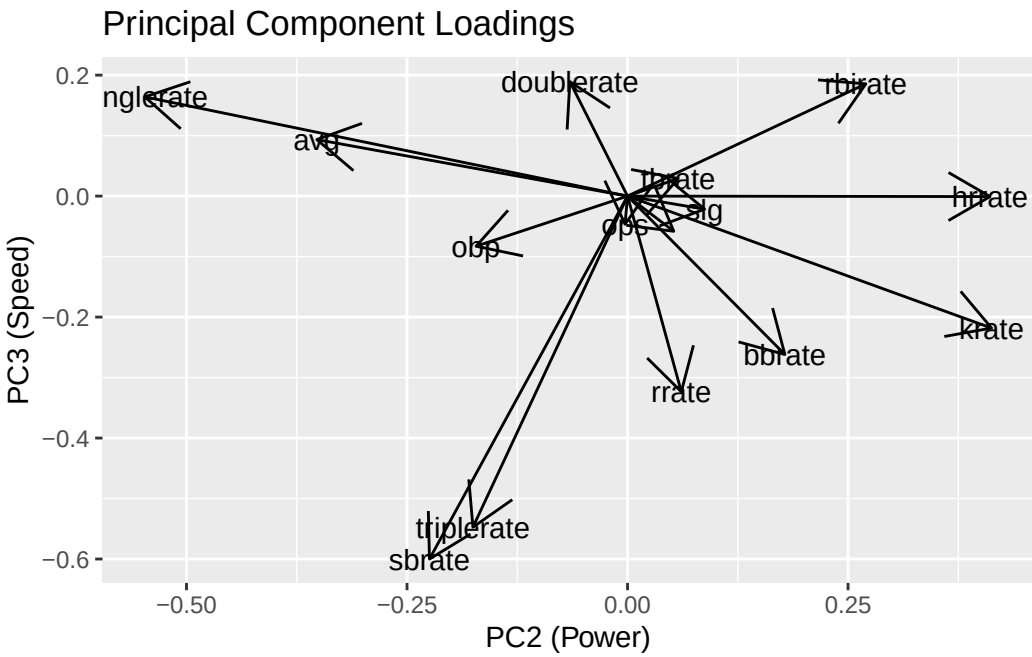
```
batting_loadings <- loadings |> pivot_wider(
  names_from="component",
  values_from="value"
) |>
  dplyr::select(!id)

ggplot(data = batting_bake,
             aes(x = PC2, y = PC3)) +
  geom_point() +
  geom_text(
    aes(label = Name),
    check_overlap = TRUE) +
  labs(
    title = "Principal Component Scores",
```

```
       x = "PC2 (Power)",
       y = "PC3 (Speed)")
```




```
ggplot(
   data = batting_loadings) +
 geom_segment(aes(
   x = 0, y = 0,
   xend = PC2, yend = PC3
 ),
 arrow = arrow(type = "open")
 ) +
 geom_text(
   aes(x = PC2,
       y = PC3,
       label = terms),
   check_overlap = TRUE
) +
 labs(
   title = "Principal Component Loadings",
    x = "PC2 (Power)",
    y = "PC3 (Speed)")
```



Principal component analysis aims to simplify a data set without losing too much important information. In doing so, it combines variables related to each other to better explain how each variable contributes to the model. Thus instead of working with close to 20 variables in this instance, PCA allows us to work with only 5 components without losing too much information about the data.

Section 4: Supervised Learning

Linear Discriminant Analysis: LDA works by first estimating $\mu_K, \pi_K,\ \Sigma$. μ_K represents the mean of the kth class. π_K denotes the prior probability that an observation belongs to the kth class. Σ represents the $p \times p$covariance matrix of X, which is shorthand for our set of predictors. Next, LDA plugs the estimates into the following equation:

$$\delta_k(x) = x^T\Sigma^{-1}\mu_k - \frac{1}{2}\mu_k^T\Sigma^{-1}\mu_k + \log \pi_k$$

to obtain $\hat{\delta}_k(x)$ and find out for which class k $\hat{\delta}_k(x)$ is largest. Note that $\delta_k(x)$ is linear in x which is the reason for linear in LDA. The assumptions made by LDA when we have p>1 predictors (which pertains to my data) are as follows:

1. $X = (X_1, X_2, \dots, X_p)$ are drawn from a multivariate normal distribution with mean vectors specific to each class, μ_K, and a common covariance matrix, Σ.

2. Due to the multivariate normal assumption, we must also assume that each predictor must follow a one-dimensional normal distribution, where some collinearity is expected between some variables.

I will also add that I tried producing a model using quadratic discriminant analysis before using LDA. However, it failed due to the covariance matrices being singular and thus the matrices did not have an inverse. I believe this happened because baseball data is heavily collinear since most statistics are made up of one another. For example, runs-batted in involves runs and at bats and OPS involves OBP and batting average. Because LDA assumes that there already exists some collinearity between variables, the covariance matrix was invertable. Even though a LDA model was produced, R still outputted warnings that told me variables are collinear. But for the sake of my curiosity, I sacrificed slight instability leaving all variables in the model in order to see how each one contributed.

```
bat_lda_model <- discrim_linear(mode="classification")|> set_engine("MASS")
bat_lda_recipe <- recipe(
  AllStar ~ age + ab + pa  + single + double + triple + hr + k + bb + avg + slg + obp+ ops +
)
bat_lda_wflow <- workflow() |> add_model(bat_lda_model) |> add_recipe(bat_lda_recipe)
bat_lda_fit <- fit(bat_lda_wflow,data=bat_train)
```

```
Warning in lda.default(x, grouping, ...): variables are collinear
```

```
bat_lda_pred <- augment(bat_lda_fit,new_data=bat_test)
bat_lda_pred
```

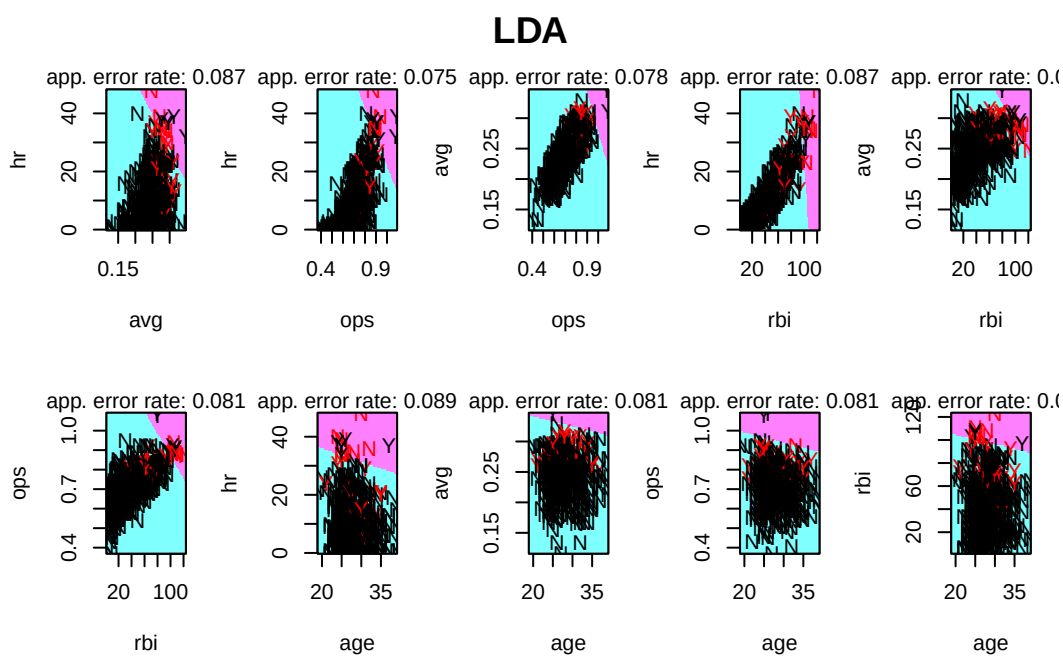
```
# A tibble: 90 x 26
   .pred_class .pred_No .pred_Yes Name   player_id  year   age    ab    pa   hit
   <fct>          <dbl>     <dbl> <chr>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
 1 No             0.999  0.000847 grega~    656185  2018    25   265   291    68
 2 No             0.984  0.0164   markt~    444432  2018    32   330   358    86
 3 No             0.999  0.000726 yange~    500208  2018    30   468   506   106
 4 No             0.998  0.00221  toddf~    453943  2018    32   408   472    87
 5 No             0.903  0.0975   edwin~    429665  2018    35   500   578   123
 6 No             1.000  0.000165 victo~    622682  2018    23   212   219    47
 7 No             0.996  0.00387  wilmy~    571976  2018    27   312   343    79
 8 No             0.997  0.00284  jorge~    624585  2018    26   223   257    59
 9 No             0.999  0.000991 delin~    592261  2018    25   334   393    72
10 No             1.000  0.000439 caleb~    543376  2018    32   265   280    58
# i 80 more rows
# i 16 more variables: single <dbl>, double <dbl>, triple <dbl>, hr <dbl>,
```

```
#   k <dbl>, bb <dbl>, avg <dbl>, slg <dbl>, obp <dbl>, ops <dbl>, rbi <dbl>,
#   tb <dbl>, sb <dbl>, g <dbl>, r <dbl>, AllStar <fct>
```

```r
conf_mat(
  bat_lda_pred,
  truth=AllStar,
  estimate=.pred_class)
```

```
          Truth
Prediction No Yes
       No  79   3
       Yes  3   5
```

```r
partimat(
  AllStar ~ hr + + avg + ops + rbi + age,
  data=bat_train,
  method="lda",
  main="LDA",
  nplots.hor=5
)
```



The hyperparameters being tuned in the chunk below are:

neighbors: the number of most similar data points from the training set that will be used to make predictions on the test set.

dist_power: the order of the Minkowski Distance. Minkowski distance is a generalization of the L1/L2 norms.This is how k-nearest neighbors controls the distance metric.

Having too few neighbors leads to a model with very high variance due to the decision boundaries becoming wiggly and overfitting.Having too many neighbors can cause the model to be a lot more stable at the cost of missing key information about the data and thus underfitting can occur.

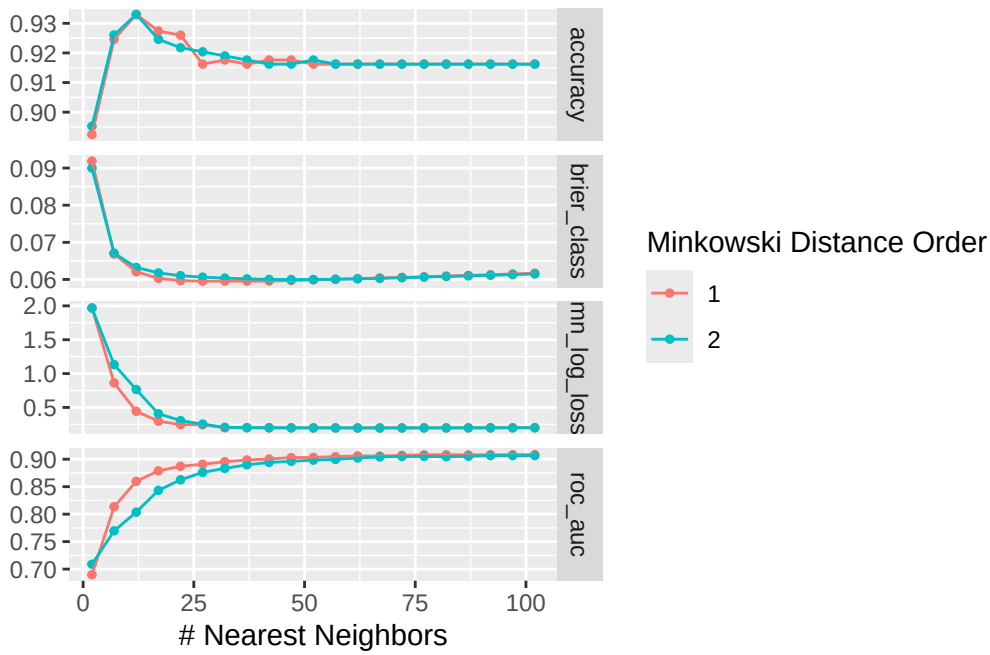
The Minkowski distance is represented as:

$$d(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p}$$

As p increases, the greater the restraint on how distance is measured in the k-nearest neighbors model. If p=1, this is the special case of Mankowski distance called the L1 norm, which allows each 'neighbor' to contribute more evenly and distance grows more slowly. Hence the model will have a slightly higher bias than variance. If p=2, we have what is called the L2 norm, and this will make larger distances between data points matter significantly more. Thus points closer to the chosen data point in the training set dominate leading to a higher variance.

```
set.seed(112)
bat_kfold <- vfold_cv(
  bat_train,
  v=5,repeats=2
)
bat_knn_model <- nearest_neighbor(
  mode="classification",
  neighbors=tune(),
  dist_power=tune()
)
bat_knn_recipe <- recipe(
  AllStar ~ age + ab + pa  + single + double + triple + hr + k + bb + avg + slg + obp+ ops +
) |>
  step_normalize(all_numeric_predictors())
bat_param_grid <-expand.grid(
  neighbors=seq(2,102,by=5),
  dist_power=c(1,2)
)
bat_knn_wflow <- workflow() |> add_model(bat_knn_model) |> add_recipe(bat_knn_recipe)
bat_tune <- bat_knn_wflow |>
  tune_grid(
```

```
    resamples=bat_kfold,
    grid=bat_param_grid,
    metrics=metric_set(roc_auc,brier_class,mn_log_loss,accuracy)
  )
autoplot(bat_tune)
```



K-Nearest Neighbors: Since the Bayes classifier is almost impossible to use in real-world applications, KNN offers an approach to estimate the conditional distribution of a response variable Y given predictors X. Given a positive integer K and an observation from the test set x_0, the algorithm finds the K closest points in the training set to x_0. These point are denoted as N_0. Then KNN estimates the conditional probability of the jth class as the fraction of points in N_0 whose response values equal j. These probabilities are calculate using the following equation:

$$P(Y = j \mid X = x_0) = \frac{1}{K} \sum_{i \in N_0} I(y_i = j)$$

where $I(y_i = j)$ is an indicator, (equals 1 if observation i is in class j, 0 otherwise). Finally, KNN classifies x_0 to the class with the largest probability from the above equation. KNN doesn't have any true, explicit assumptions, but it does imply and is sensitive to the following:

1. Distance is a huge factor in KNN; KNN assumes points close to the aforementioned x_0 should be similar to it and vice versa.

2. Predictors must be on a similar scale otherwise large scale variables will overwhelm the model. For example, my predictor, home runs, range from 0 to 50 while the predictor batting average ranges from, let's say, .100 to .400. Thus it is imperative that variables are normalized before fitting a KNN model.

3. Having a much greater number of observations, n, than number of predictors, p. This relates to the fact that KNN is non-parametric, (doesn't have a fixed form that explicitly tells the relationship between a response variable and some predictors), and thus reduces the bias while increasing the variance.

```
bat_knn_final_model <- nearest_neighbor(
  mode="classification",
  neighbors=20,
  dist_power=1
)
bat_knn_final_wflow <- workflow() |> add_model(bat_knn_final_model) |> add_recipe(bat_knn_re
bat_knn_tuned <- fit(bat_knn_final_wflow,data=bat_train)
bat_knn_pred <- augment(bat_knn_tuned,new_data=bat_test)
bat_knn_pred
```

```
# A tibble: 90 x 26
   .pred_class .pred_No .pred_Yes Name   player_id  year   age    ab    pa   hit
   <fct>          <dbl>     <dbl> <chr>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
 1 No             1        0      grega~    656185  2018    25   265   291    68
 2 No             0.941    0.0587 markt~    444432  2018    32   330   358    86
 3 No             0.971    0.0287 yange~    500208  2018    30   468   506   106
 4 No             0.975    0.0255 toddf~    453943  2018    32   408   472    87
 5 No             0.773    0.227  edwin~    429665  2018    35   500   578   123
 6 No             1        0      victo~    622682  2018    23   212   219    47
 7 No             1        0      wilmy~    571976  2018    27   312   343    79
 8 No             1        0      jorge~    624585  2018    26   223   257    59
 9 No             1        0      delin~    592261  2018    25   334   393    72
10 No             1        0      caleb~    543376  2018    32   265   280    58
# i 80 more rows
# i 16 more variables: single <dbl>, double <dbl>, triple <dbl>, hr <dbl>,
#   k <dbl>, bb <dbl>, avg <dbl>, slg <dbl>, obp <dbl>, ops <dbl>, rbi <dbl>,
#   tb <dbl>, sb <dbl>, g <dbl>, r <dbl>, AllStar <fct>
```

```
conf_mat(
  bat_knn_pred,
  truth=AllStar,
  estimate=.pred_class
)
```

```
          Truth
Prediction No Yes
       No  82   5
       Yes  0   3
```

```r
bat_lda_results <- bat_lda_wflow |>
  fit_resamples(
    resamples = bat_kfold,
    metrics = metric_set(accuracy)
  )
```

```
> A | warning: variables are collinear

There were issues with some computations   A: x1

There were issues with some computations   A: x10
```

```r
lda_results <- bat_lda_results |>
  collect_metrics()
lda_results <- lda_results |>
  mutate(sd = std_err * sqrt(10))
lda_results
```

```
# A tibble: 1 x 7
  .metric  .estimator  mean     n std_err .config             sd
  <chr>    <chr>      <dbl> <int>   <dbl> <chr>            <dbl>
1 accuracy binary     0.918    10 0.00885 pre0_mod0_post0 0.0280
```

```r
knn_results <- bat_tune |>
  collect_metrics() |>
  filter(.metric=="accuracy")
knn_results <- knn_results |>
  mutate(sd=std_err*sqrt(10))
knn_results |> filter(neighbors==22,dist_power==1)
```

```
# A tibble: 1 x 9
  neighbors dist_power .metric  .estimator  mean     n std_err .config        sd
      <dbl>      <dbl> <chr>    <chr>      <dbl> <int>   <dbl> <chr>       <dbl>
1        22          1 accuracy binary     0.926    10  0.0106 pre0_mod0~ 0.0336
```

I used 5-fold cross validation with 2 repetitions to acquire the average accuracy and the standard deviation of average accuracy for each model. The mean accuracies are similar for each model where the LDA model has a mean accuracy of0.91768 and the KNN model has a mean accuracy of 0.925 The differences in standard error and standard deviation are negligible compared to the noise of k-fold cross validation.